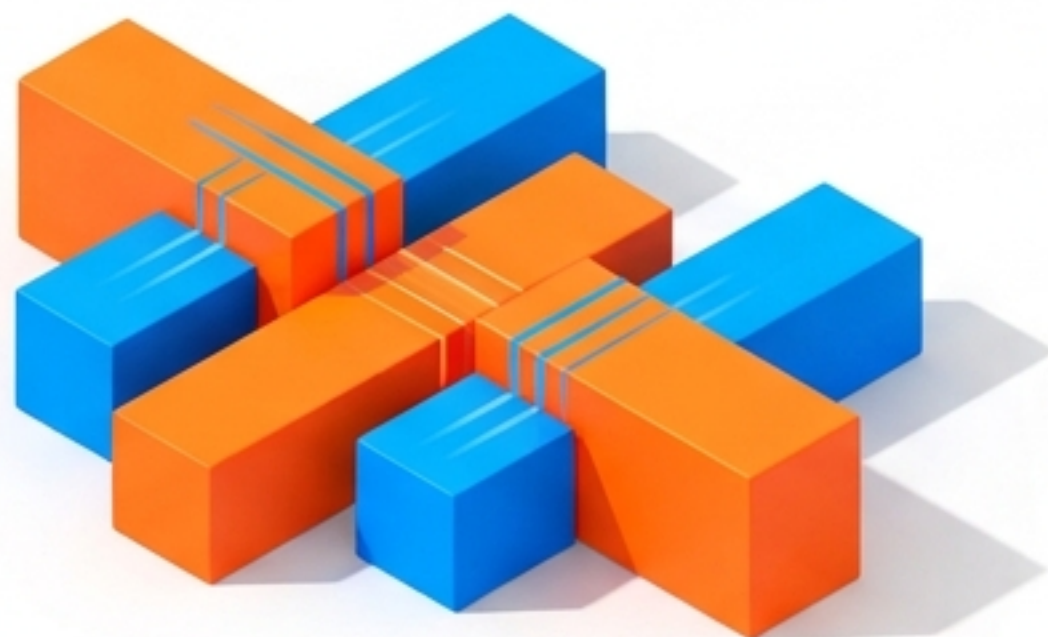


Phép toán Tensor: Xử lý theo phần tử (Element-wise)



Khái niệm

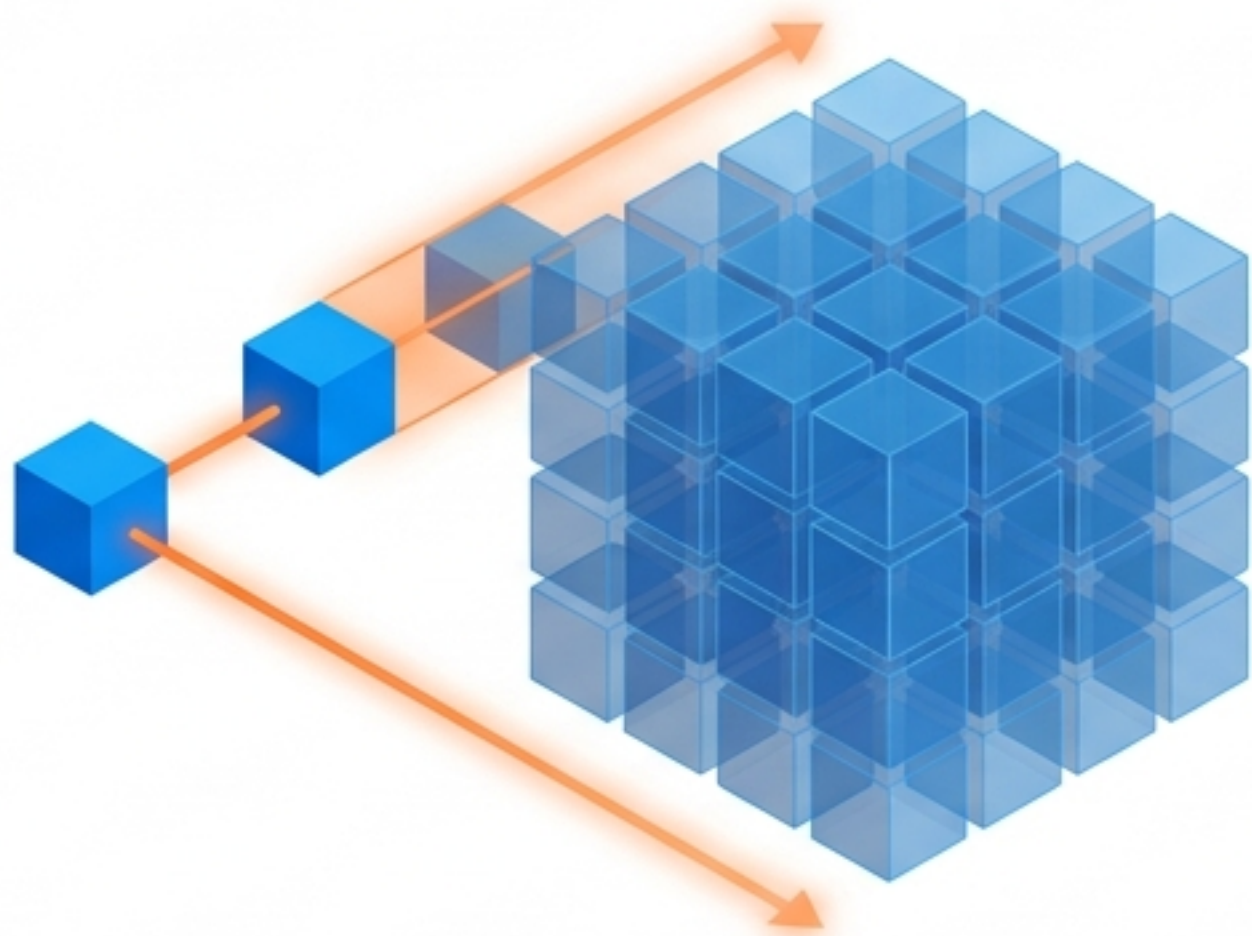
Áp dụng toán học độc lập cho từng mục bên trong tensor (Cộng, Trừ, Hàm ReLU...).



Hiệu suất

Tốc độ cực cao. Được thực thi song song toàn diện qua các thư viện BLAS siêu tốc trên kiến trúc GPU.

Cơ chế Phát sóng (Broadcasting)



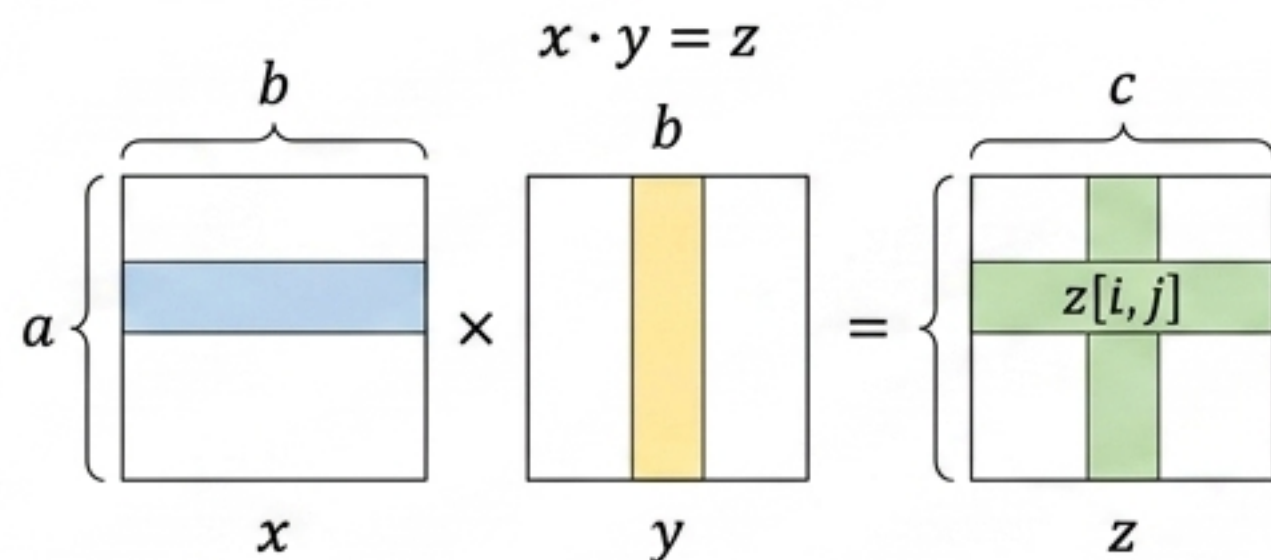
Cơ chế: Cho phép tính toán giữa 2 tensor khác kích thước (VD: Ma trận + Vector). Tensor nhỏ tự động 'lặp lại' dọc theo trục mới để khớp hình dạng của tensor lớn.

```
import numpy as np

X = np.random.random((32, 10))
y = np.random.random((10,))

Z = X + y # y tự động phát sóng thành (32, 10) rồi mới cộng.
```

Sản phẩm Tensor (Tích vô hướng - Dot Product)



Hình 2.5: Sơ đồ hộp minh họa tích ma trận

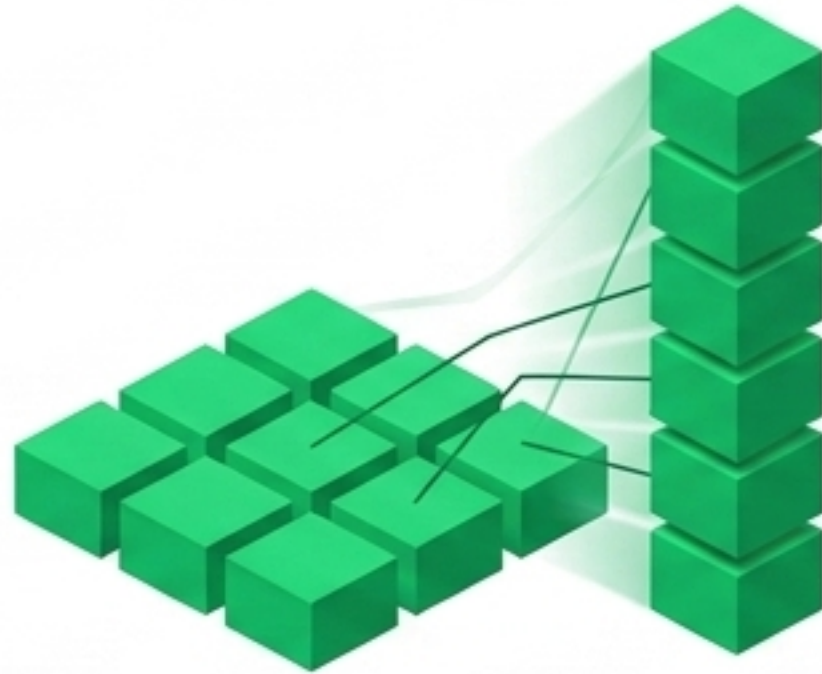
Đặc điểm: Phép toán phổ biến nhất trong mạng nơ-ron (sử dụng `matmul` hoặc toán tử `@`). Không giống phép nhân từng phần tử.

Quy tắc hình dạng: Kết hợp các hàng của ma trận 1 với cột của ma trận 2.
 $(a, b) \cdot (b, c) \rightarrow (a, c)$

Biến đổi Không gian: Reshape và Transpose

Định hình lại (Reshaping)

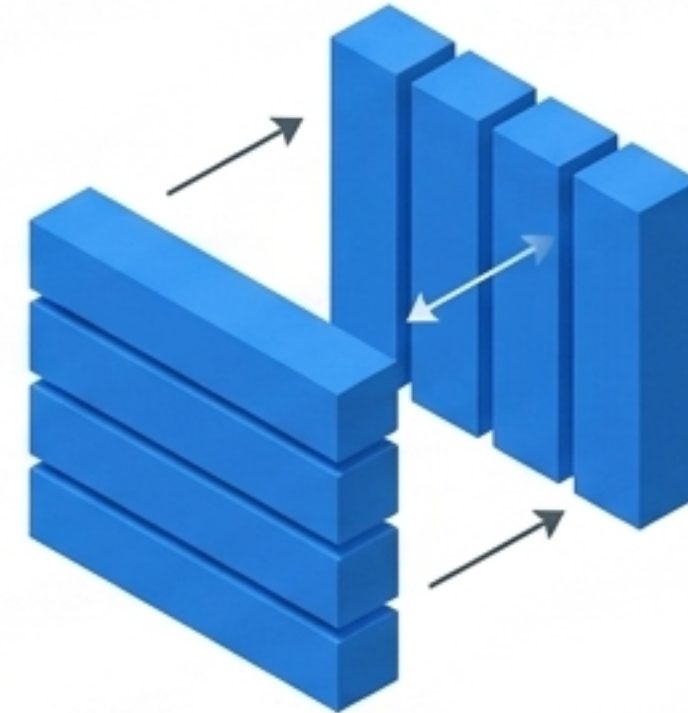
Sắp xếp lại các hàng và cột để đạt được hình dạng mục tiêu mà không thay đổi tổng số phần tử.



```
x = np.array([[0. , 1.], [2. , 3.], [4. , 5.]]) # Shape: (3, 2)
x = x.reshape ((6, 1)) # Trở thành cột 6 phần tử
```

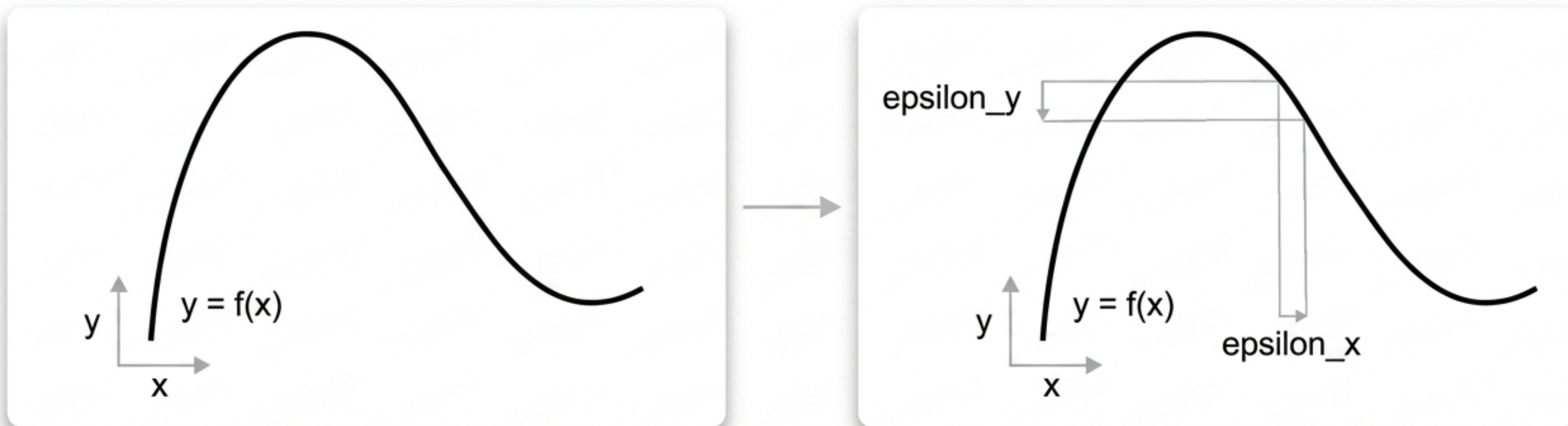
Chuyển vị (Transposing)

Tráo đổi trực tiếp hàng thành cột.



```
x = np.zeros ((300 , 20))
x = np.transpose(x) # Shape trở thành (20, 300)
```

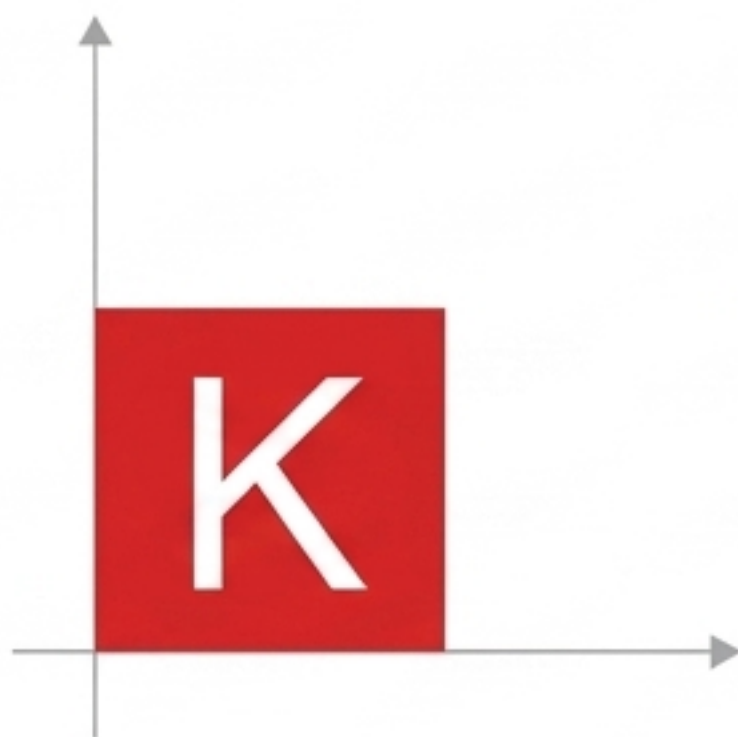
Góc nhìn Hình học: Mạng Nơ-ron là một Hàm số



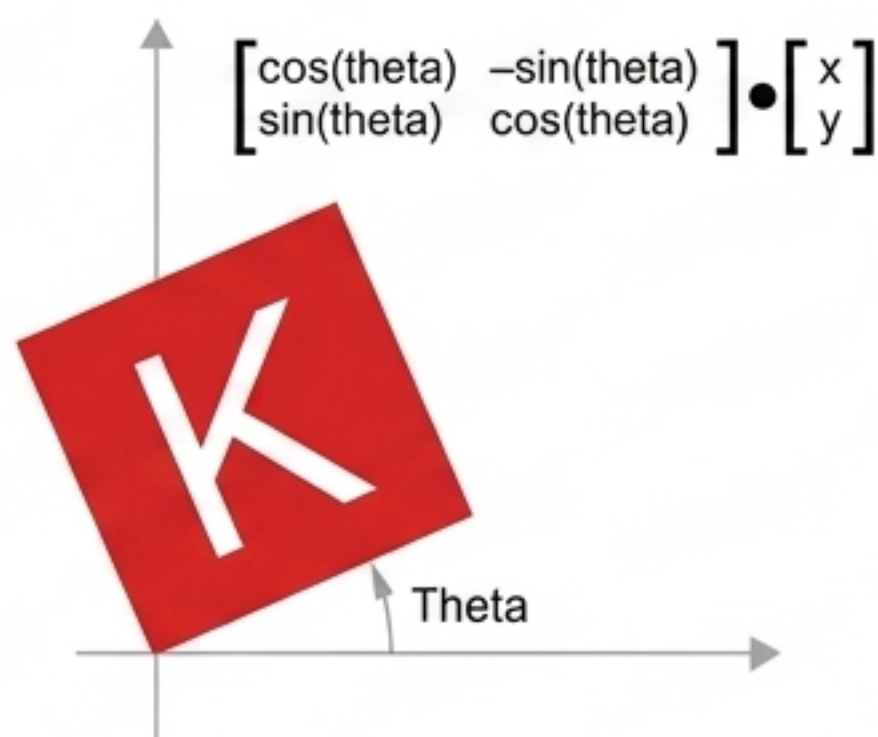
Tất cả phép toán tensor đều mang ý nghĩa biến đổi hình học.

Bản chất của một mạng nơ-ron thực chất là một hàm toán học khổng lồ, yêu cầu tính liên tục và trơn tru.

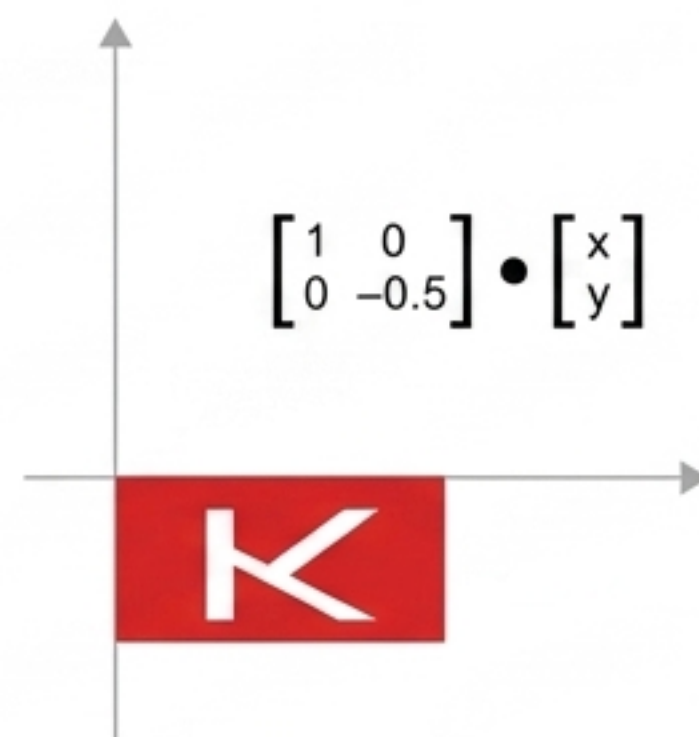
Khảo sát các Phép Biến đổi Hình học Cơ bản



Hình 2.7: Tịnh tiến: Dời hình bằng phép cộng vector.

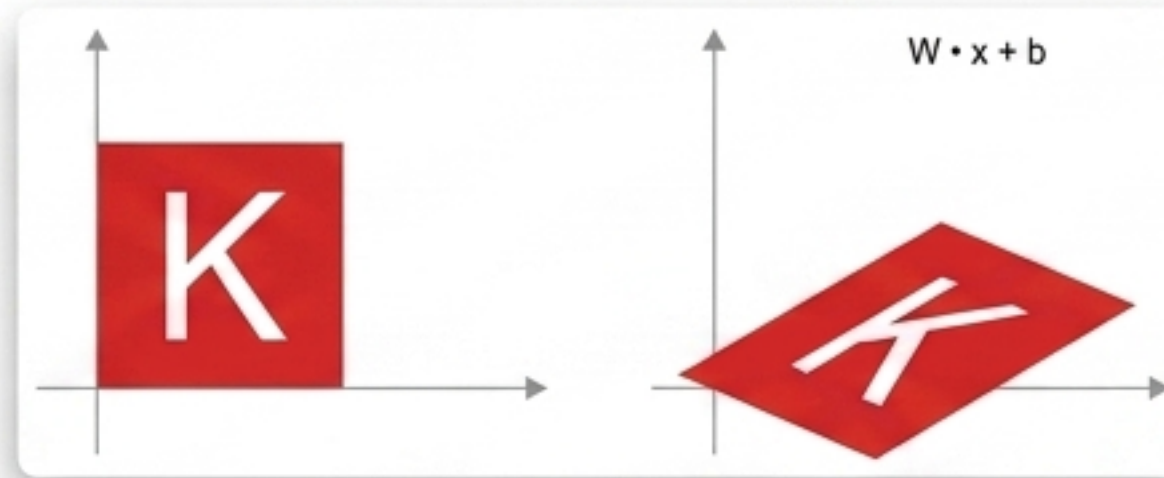


Hình 2.8: Quay: Tích ma trận 2×2 (Lượng giác).

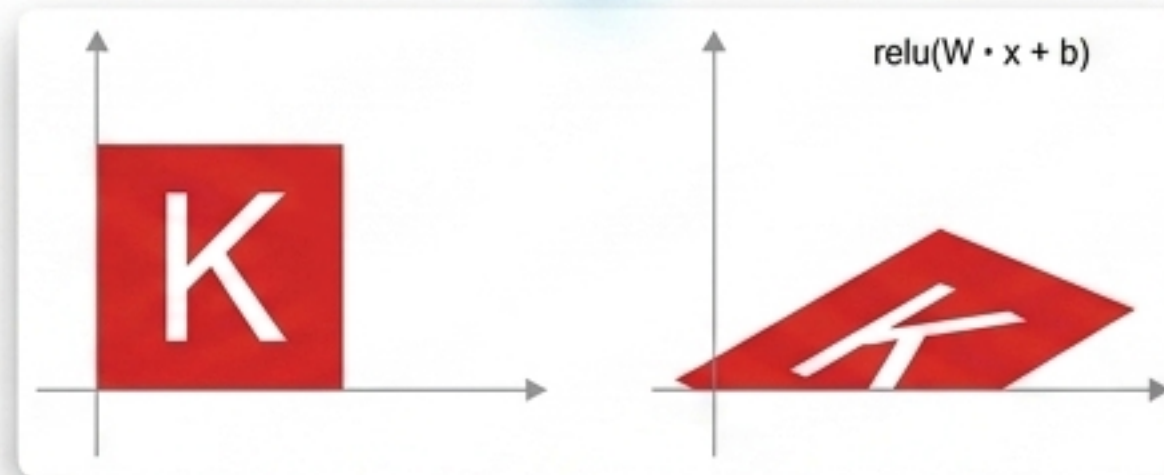


Hình 2.9: Thu phóng: Tích ma trận đường chéo.

Lớp Dense & Kích hoạt Phi tuyến (ReLU)



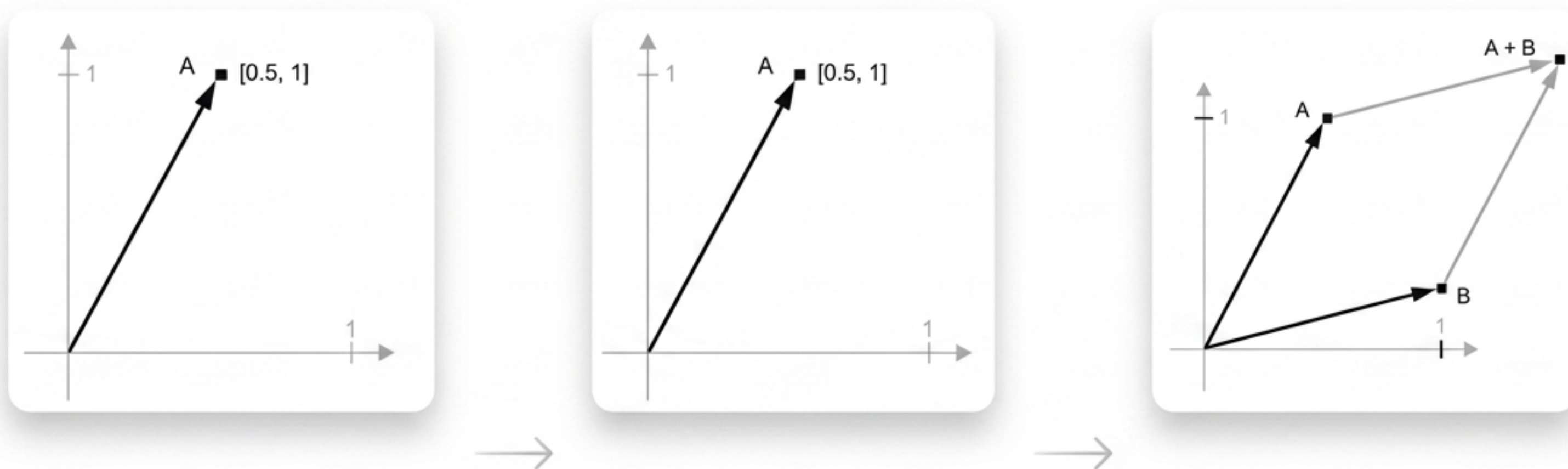
Lớp Dense (Affine): Sự kết hợp của biến đổi tuyến tính (nhân trọng số W) và tịnh tiến (cộng bias b). Công thức: $y = W \cdot x + b$.



Kích hoạt phi tuyến (ReLU): Bóp méo không gian affine, tạo ra các ranh giới phức tạp cần thiết để phân tách dữ liệu đa chiều.

Hình 2.10: Biến đổi Affine & theo sau là ReLU

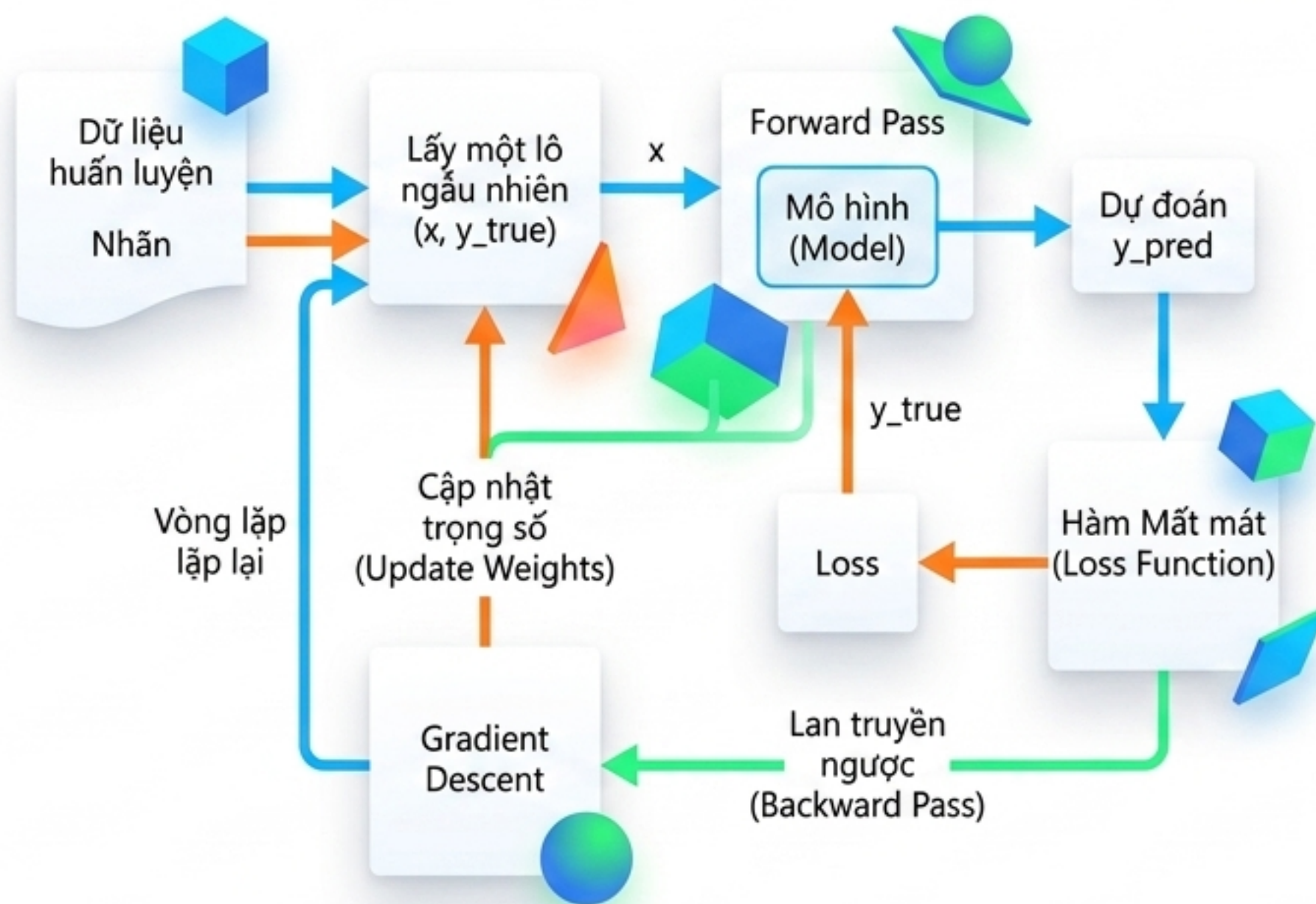
Mục đích tối thượng: Gỡ rối Dữ liệu



Hình 2.11: Quá trình phân tách dữ liệu qua các lớp ẩn giống như việc từ từ vuốt phẳng một quả bóng giấy vò nát.

Học sâu bản chất là một chuỗi các phép biến đổi hình học tinh vi.
Mục tiêu: "Gỡ rối" (*untangle*) các đa tạp dữ liệu phức tạp đan xen nhau thành các không gian phân tách rõ ràng.

Hệ sinh thái Đào tạo: Vòng lặp Học (Training Loop)



Khởi tạo ngẫu nhiên trọng số mô hình W và b .

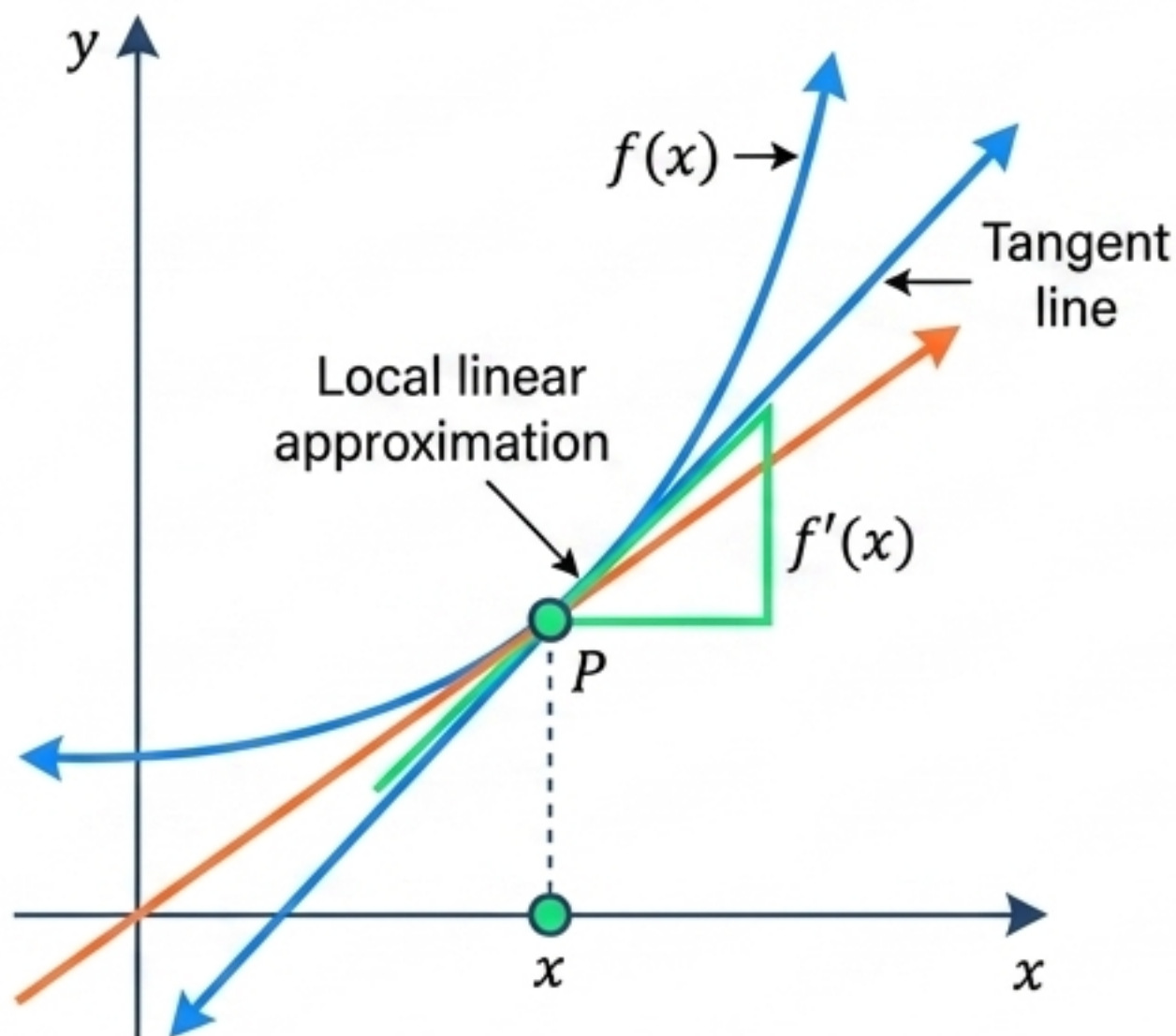
1. Lấy một lô ngẫu nhiên dữ liệu huấn luyện x và nhãn y_{true} .

2. Forward Pass: Chạy mô hình trên x để dự đoán y_{pred} .

3. Tính hàm Mất mát (Loss) để đo lường sai số dự đoán.

4. Cập nhật trọng số: Điều chỉnh tất cả trọng số theo hướng làm giảm Loss (thông qua **Gradient Descent**).

Chìa khóa Tối ưu: Khái niệm Đạo hàm (Derivative)



Hình 2.12: Đạo hàm là hệ số góc của tiếp tuyến

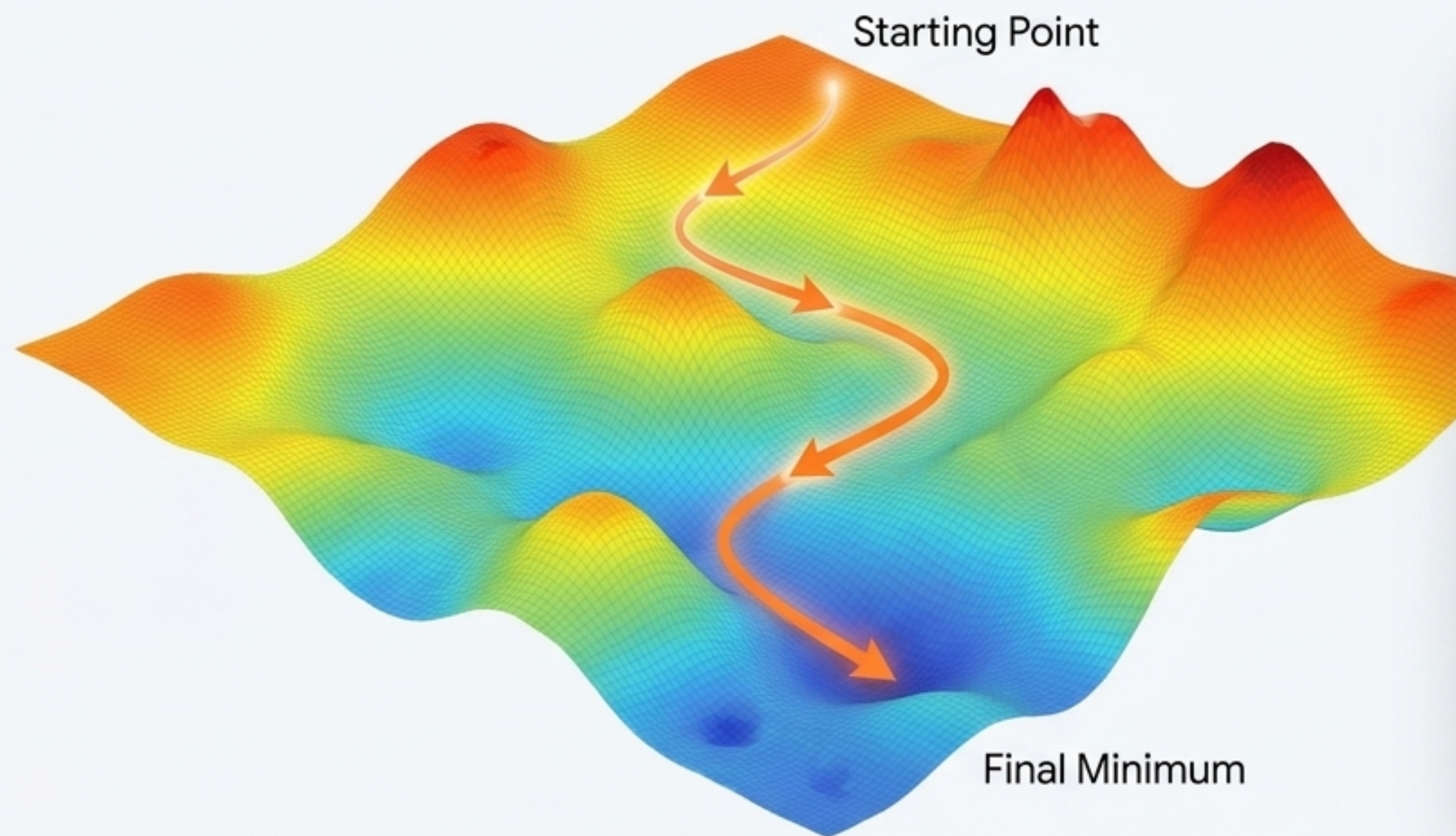
Đạo hàm $f'(x)$ thể hiện độ dốc (slope) của tiếp tuyến với hàm số $f(x)$ tại điểm x .

Quy tắc cập nhật:

- Nếu $f'(x) > 0$, tăng x sẽ làm hàm tăng.
- Nếu $f'(x) < 0$, tăng x sẽ làm hàm giảm.

Kết luận: Muốn giảm giá trị Loss, hệ thống chỉ cần đi ngược hướng với đạo hàm.

Gradient: Mặt cong Đa chiều của Loss



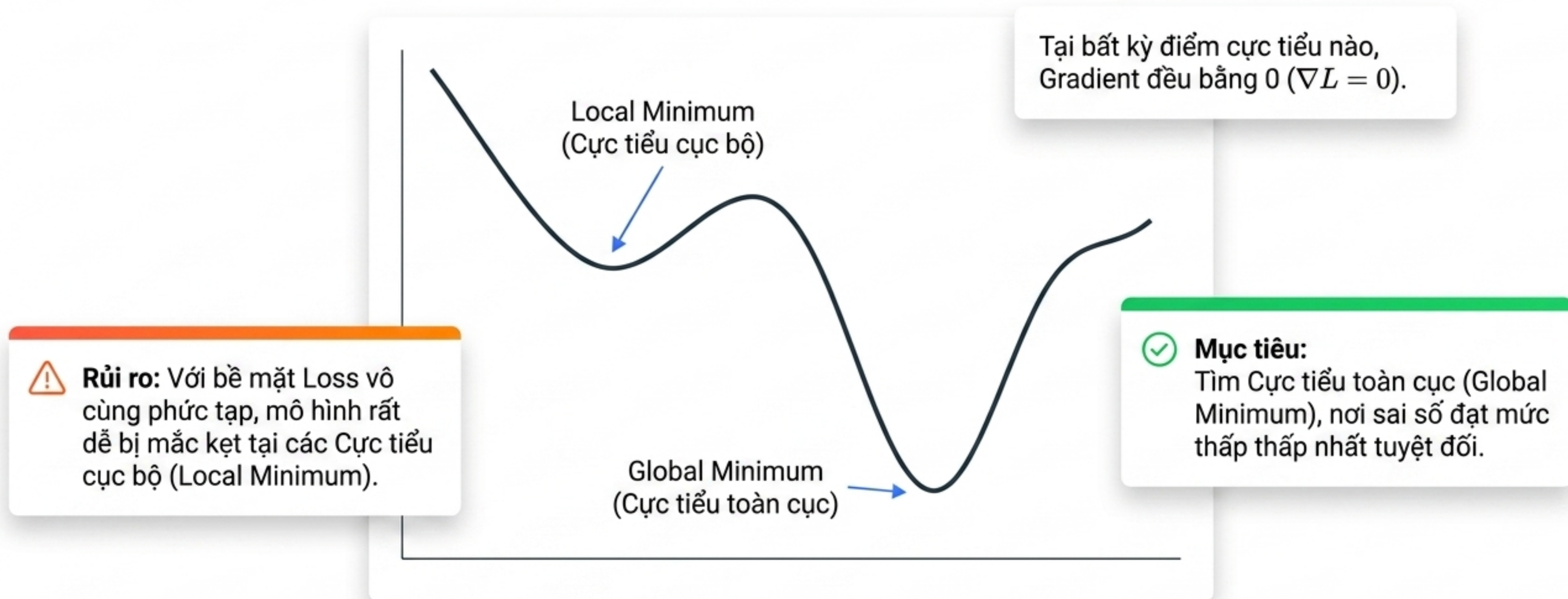
Gradient: Sự tổng quát hóa của đạo hàm trong không gian đa chiều chứa hàng triệu tham số. Nó biểu thị độ cong của bề mặt hàm Loss.

Vectơ Gradient luôn chỉ về hướng hàm L tăng nhanh nhất.

Hành trình Tối ưu: Đi ngược hướng Gradient chính là con đường ngắn nhất để trượt xuống 'thung lũng' Loss.

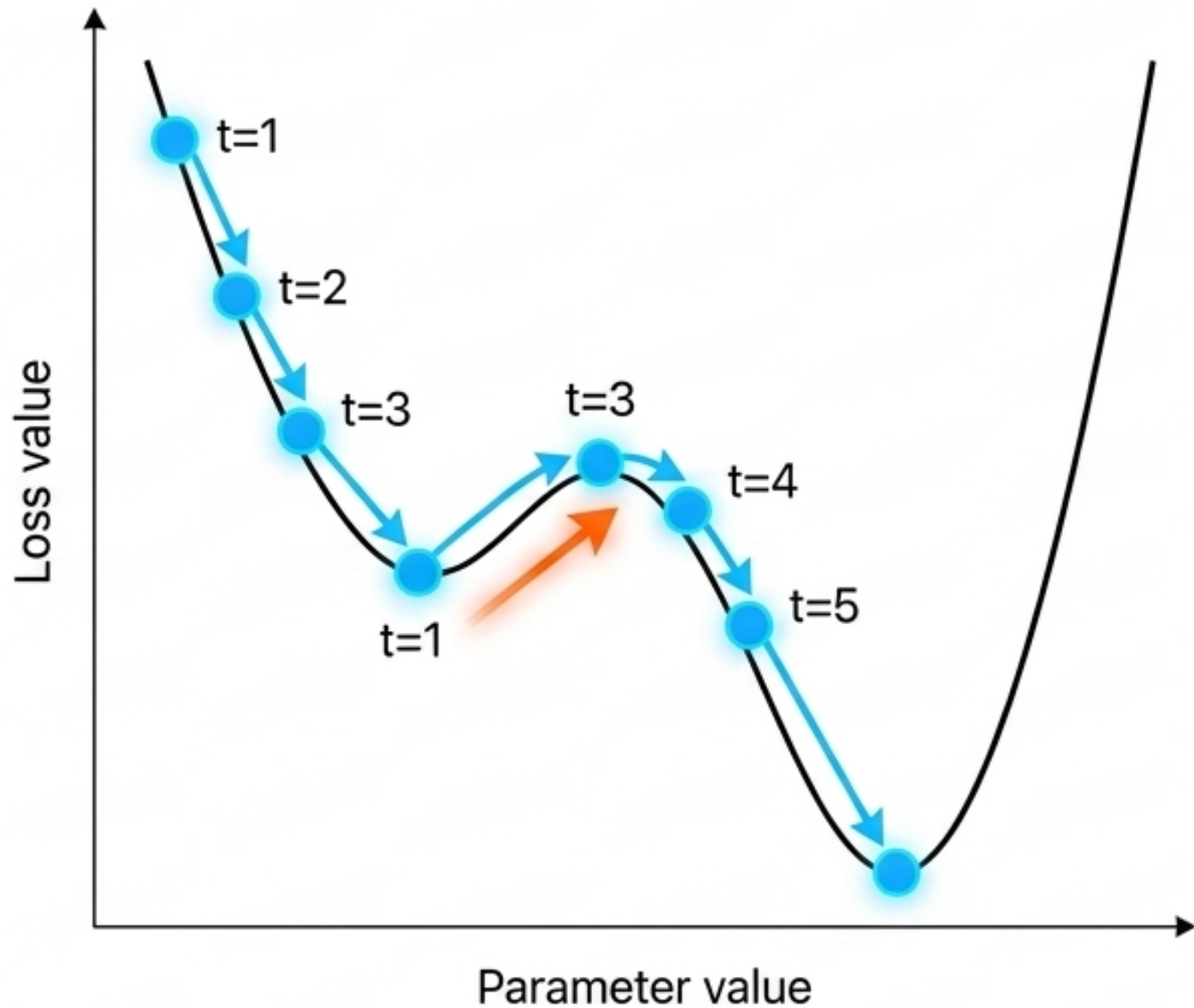
Hình 2.13: Mặt cong 3D của Loss function

Bầy Tối ưu: Cực tiểu Toàn cục và Cục bộ



Hình 2.14: Local Minimum vs Global Minimum

Thuật toán Tối ưu: SGD và Động lượng (Momentum)



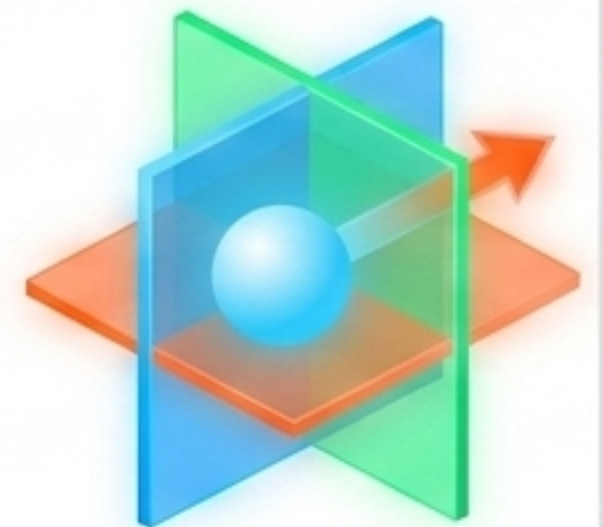
Stochastic Gradient Descent (SGD)

Cập nhật trọng số dựa trên một mẻ nhỏ ngẫu nhiên.

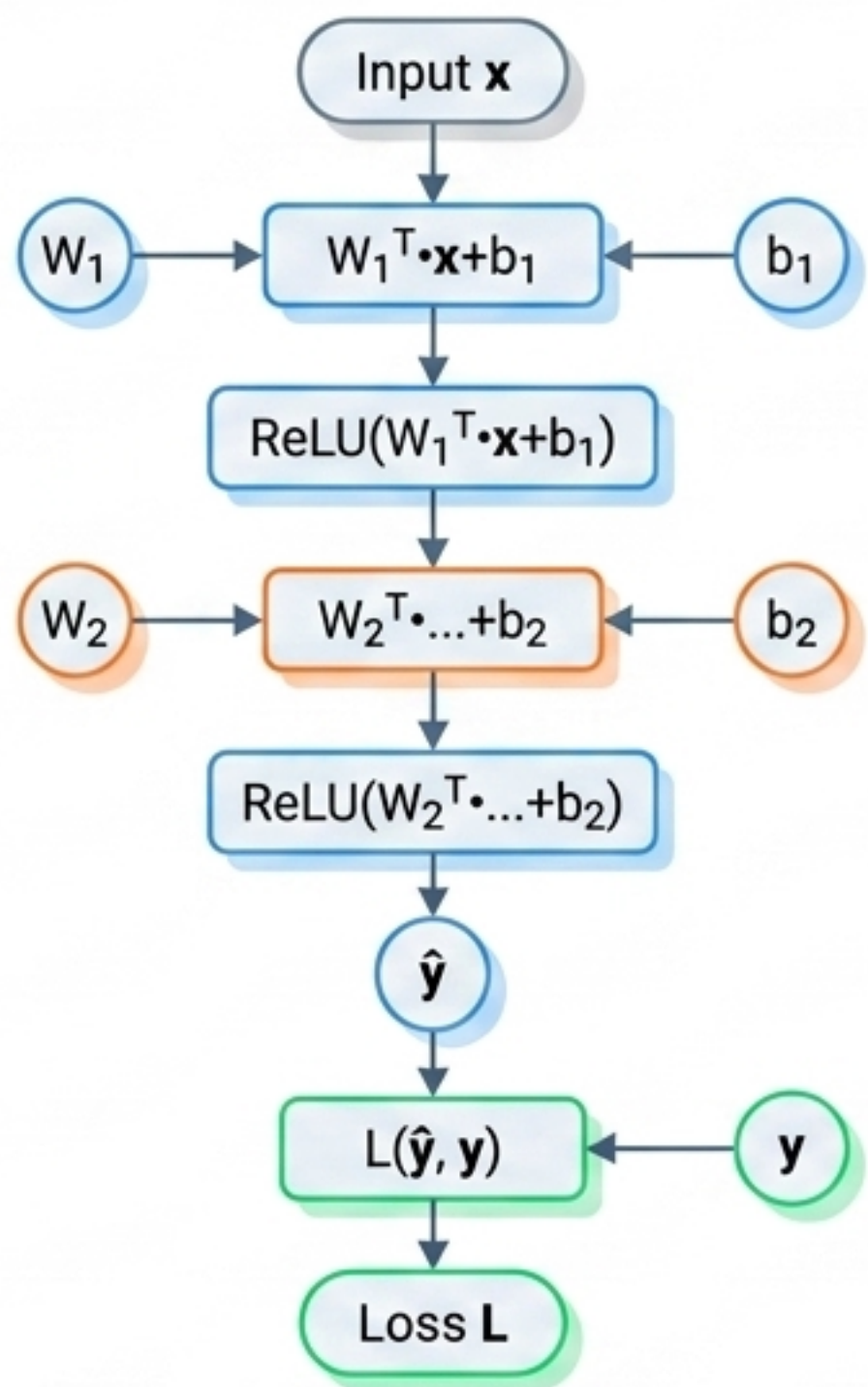
$$\text{Công thức: } W \leftarrow W - (\text{learning_rate} \times \nabla L)$$

Động lượng (Momentum)

Lấy cảm hứng từ vật lý (quả bóng lăn xuống dốc). Sử dụng vận tốc từ quá khứ để gia tốc quả bóng, giúp nó có đủ 'đà' để vọt qua và thoát khỏi các cực tiểu cục bộ hẹp.



Kiến trúc Hộp Đen: Đồ thị Tính toán (Computation Graph)



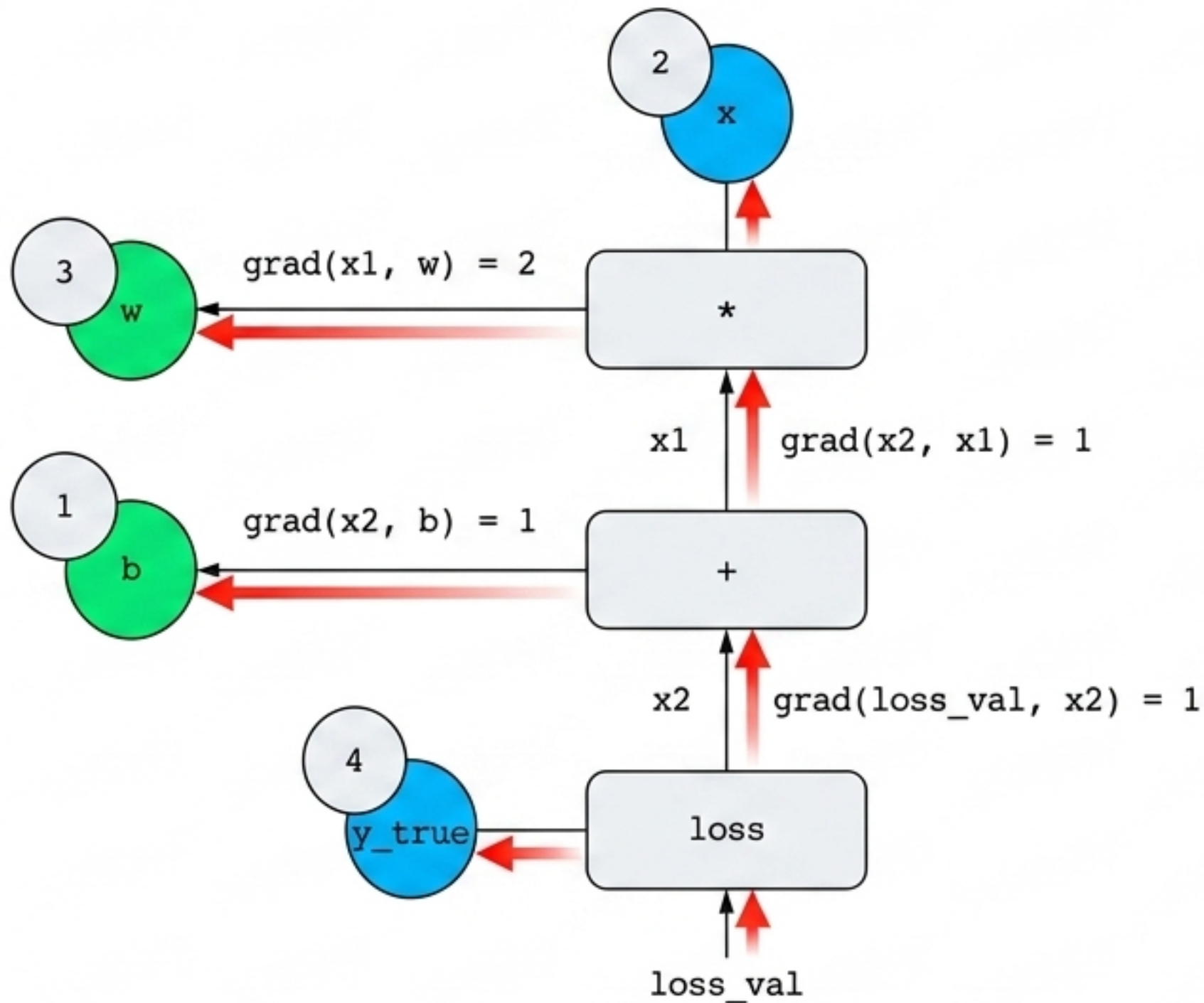
Bài toán: Làm sao để tính đạo hàm của hàng triệu trọng số cùng lúc?

Giải pháp: Cốt lõi của mô hình là Đồ thị tính toán.

Cơ chế: Biến các phép toán Tensor thành các 'nút' (node) và dòng dữ liệu thành các 'cạnh'. Cấu trúc hóa toán học thành dữ liệu máy tính.

Hình 2.16: Biểu diễn đồ thị cho mô hình mạng nơ-ron

Lan truyền ngược & Quy tắc Chuỗi (Chain Rule)



Lan truyền ngược (Backpropagation):

Dòng chảy thông tin đi ngược từ Loss function lên các layer đầu tiên để tính toán Gradient cho từng trọng số.

Quy tắc Chuỗi: Nền tảng toán học cho phép phân rã và tính đạo hàm của các hàm hợp phức tạp.

Công thức:

$$\frac{d(f(g(x)))}{dx} = f'(g(x)) \cdot g'(x)$$